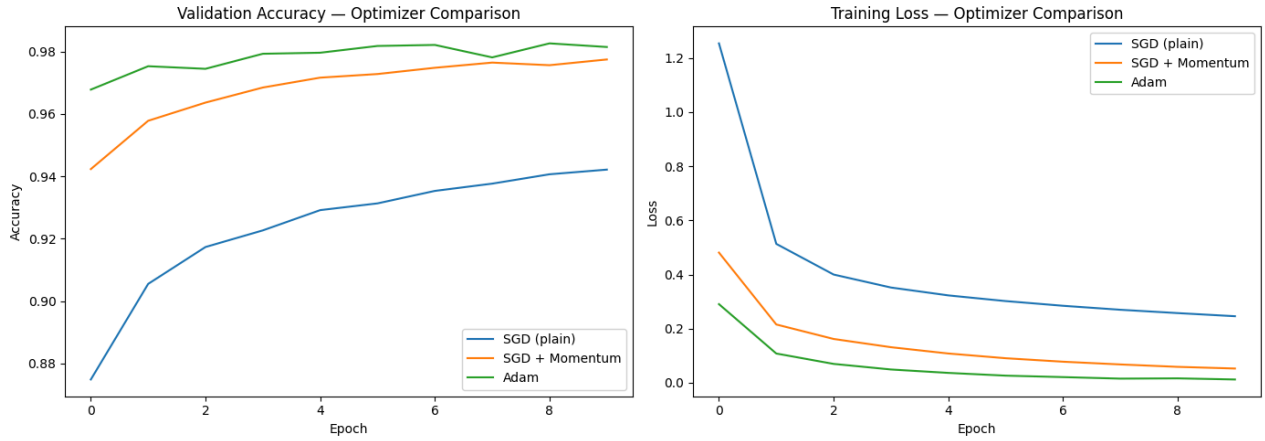
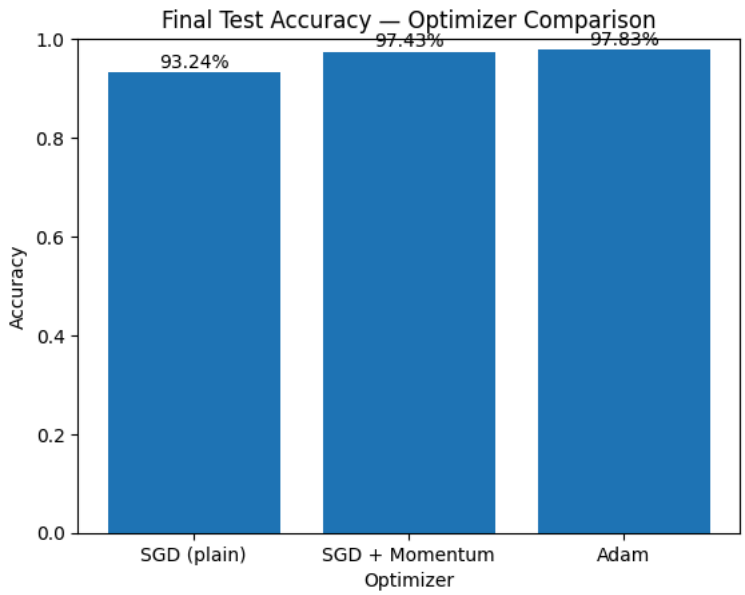
```
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.optimizers import SGD, Adam

# Load & preprocess
(X_train, y_train), (X_test, y_test) = mnist.load_data()
X_train = X_train.reshape(60000, 784).astype('float32') / 255.0
X_test  = X_test.reshape(-1, 784).astype('float32')  / 255.0
y_train_cat = to_categorical(y_train)
y_test_cat  = to_categorical(y_test)

def build_model():
    model = Sequential()
    model.add(Dense(256, input_dim=784, activation='relu'))
    model.add(Dense(128, activation='relu'))
    model.add(Dense(10,  activation='softmax'))
    return model

# Three optimizers to compare
optimizers = {
    'SGD (plain)':     SGD(learning_rate=0.01),
    'SGD + Momentum':  SGD(learning_rate=0.01, momentum=0.9),
    'Adam':            Adam(learning_rate=0.001)
}

histories = {}
results   = {}

for name, optim in optimizers.items():
    print(f"\nTraining with: {name}")
    model = build_model()
    model.compile(loss='categorical_crossentropy',
                  optimizer=optim,
                  metrics=['accuracy'])
    h = model.fit(X_train, y_train_cat,
                  batch_size=128, epochs=10,
                  validation_split=0.1, verbose=0)
    histories[name] = h
    test_acc = model.evaluate(X_test, y_test_cat, verbose=0)[1]
    results[name] = test_acc
    print(f"  Test Accuracy: {test_acc*100:.2f}%")

# Plot comparison
plt.figure(figsize=(14, 5))

plt.subplot(1, 2, 1)
for name, h in histories.items():
    plt.plot(h.history['val_accuracy'], label=name)
plt.title('Validation Accuracy — Optimizer Comparison')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()

plt.subplot(1, 2, 2)
for name, h in histories.items():
    plt.plot(h.history['loss'], label=name)
plt.title('Training Loss — Optimizer Comparison')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()

plt.tight_layout()
plt.show()

# Summary table
print("\n── Final Test Accuracy ──")
for name, acc in results.items():
    print(f"  {name:20s}: {acc*100:.2f}%")

# Bar graph for final test accuracy comparison
plt.figure()
names = list(results.keys())
accuracies = list(results.values())

plt.bar(names, accuracies)
plt.title('Final Test Accuracy — Optimizer Comparison')
plt.xlabel('Optimizer')
```

```
plt.ylabel('Accuracy')
plt.ylim(0, 1)

# Show values on top of bars (looks good in reports)
for i, v in enumerate(accuracies):
    plt.text(i, v + 0.01, f"{v*100:.2f}%", ha='center')

plt.show()
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step

Training with: SGD (plain)
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape`/`input
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
  Test Accuracy: 93.24%

Training with: SGD + Momentum
  Test Accuracy: 97.43%

Training with: Adam
  Test Accuracy: 97.83%
```




```
── Final Test Accuracy ──
  SGD (plain)          : 93.24%
  SGD + Momentum       : 97.43%
  Adam                 : 97.83%
```



Start coding or generate with AI.